Runway Foreign Object Debris (FOD) Detection

Technical Report & UAV Edge Architecture

Author: Individual Project Submission | **Focus:** Computer Vision, Runway Safety & UAV Deployment | **Validation:** Held-Out Test Set (mAP@50: 96.81%, Recall: 95.77%)

Runway Foreign Object Debris (FOD) Detection

Technical Report & UAV Edge Architecture

Author: Individual Project Submission
Focus: Computer Vision, Runway Safety & UAV Deployment

1. Project Overview

Foreign Object Debris (FOD) on runways, such as bolts, tools, tire chunks, and rocks, creates serious hazards for aircraft during takeoff and landing.

This project builds an automated detection system using a lightweight YOLO model suitable for drone inspections. The model is tuned to prioritize high recall so dangerous debris is not missed.

2. Dataset and Preprocessing

2.1 Dataset Selection (FOD-A)

I used the FOD-A (Foreign Object Debris in Airports) dataset.

- Real airport tarmac background with actual runway surface textures, painted markings, and joint lines.
- Diverse environmental conditions: dry pavement (78.6%), wet pavement with puddles (21.4%), bright sun (50.1%), overcast lighting (36.9%), and dusk (13.0%).
- Real aerial camera angles representing drone inspection altitudes.

2.2 Class Mapping (31 to 6 Classes)

The original dataset has 31 granular categories. Many categories have fewer than 15 samples. I grouped them into 6 functional classes matching airport response procedures (FAA AC 150/5210-24):

1. **fastener:** Bolts, nuts, screws, washers, rivets, nails (puncture risk).
2. **tool:** Wrenches, pliers, cutters, screwdrivers, hammers (airframe impact risk).
3. **tire_fragment:** Shredded tire rubber and hoses (engine and fuel tank risk).
4. **metal_scrap:** Wires, sheet metal, cans, brackets, fuel caps (ingestion risk).
5. **plastic_debris:** Tags, tape, pens, plastic fragments (sensor clogging risk).
6. **stone_rock:** Chipped asphalt, concrete ballast, rocks (turbine erosion risk).

2.3 Data Splitting and Scale

- Small objects dominate: 54.2% of all objects are under 32x32 pixels, 37.1% are medium, and 8.7% are large.
- Dataset was curated to 4,000 diverse images to avoid redundant video frames.
- Stratified Split: 70% Train (2,800 images), 15% Validation (600 images), 15% Test (600 images).

3. Model Architecture and Training Setup

3.1 Model Choice: YOLO11 Nano (`yolo11n`)

I selected YOLO11 Nano for drone edge deployment:

- **Spatial Attention (C2PSA):** Runways are mostly uniform asphalt. C2PSA helps the model focus on small anomalous objects like bolts and shiny scrap without wasting capacity on empty tarmac.
- **Lightweight:** Only 2.59M parameters and 6.4 GFLOPs (22% smaller than YOLOv8n).
- **Fast:** Runs at 4.3 ms per image (~150+ FPS) on an RTX 2050 GPU, and around 14 ms (65+ FPS) on a Jetson Orin Nano.
- **Compared to other options:** Faster R-CNN (41M params, >85 ms) is too slow for real-time drones, and RT-DETR (32M params) requires too much power.

3.2 Hyperparameters

- **Input Size:** 640x640 (preserves small objects below 32 pixels).
- **Batch Size:** 16 with FP16 automatic mixed precision.
- **Epochs:** 35 epochs using AdamW optimizer with cosine learning rate schedule (lr: 0.002 to 0.00002).
- **Augmentation:** Mosaic, HSV color jitter, and rotation/flips to simulate different drone altitudes and lighting.

4. Test Set Evaluation (600 Images)

Evaluated on the 600 held-out test images:

Metric	Score	Note
Precision	**95.34%**	Low false alarm rate
Recall	**95.77%**	Over 95% of hazards detected
mAP@50	**96.81%**	Strong overall detection accuracy
mAP@50-95	**71.53%**	Accurate box localization

Class-Wise Test Performance

Class	Test Instances	Precision	Recall	mAP@50	mAP@50-95
fastener	171	96.7%	95.9%	97.0%	62.2%
tool	167	98.7%	93.7%	97.3%	73.2%
tire_fragment	5	100.0%	100.0%	99.5%	83.7%
metal_scrap	124	93.5%	93.0%	94.8%	68.9%
plastic_debris	113	96.7%	92.0%	96.0%	68.0%
stone_rock	33	86.4%	100.0%	96.2%	73.3%

Fasteners have a lower mAP@50-95 because tiny 15-pixel objects are sensitive to small pixel offsets in IoU, but operational detection (mAP@50) remains high at 97.0%.

5. Error Analysis and Confidence Threshold

5.1 Breakdown of Detections (600 Test Images)

- **True Positives (TP):** 595
- **False Positives (FP):** 75
- **False Negatives (FN):** 18

5.2 Why Errors Happened

1. **False Positives (False Alarms):**
   - Sun glint on asphalt aggregate (quartz granules reflect light like metal).
   - Chipped paint edges on runway and taxiway markings.
   - Black tar sealant in pavement expansion joints (can look like rubber hose pieces).
2. **False Negatives (Misses):**
   - Tiny debris smaller than 15 pixels.
   - Dark hardware sitting inside shadows or dark asphalt patches.
   - Reflections in rain puddles distorting object edges.

5.3 Runway Safety Dilemma & Threshold Selection

In airport operations, a missed object (FN) can cause engine failure or tire bursts during takeoff. A false alarm (FP) only causes a quick inspection check.

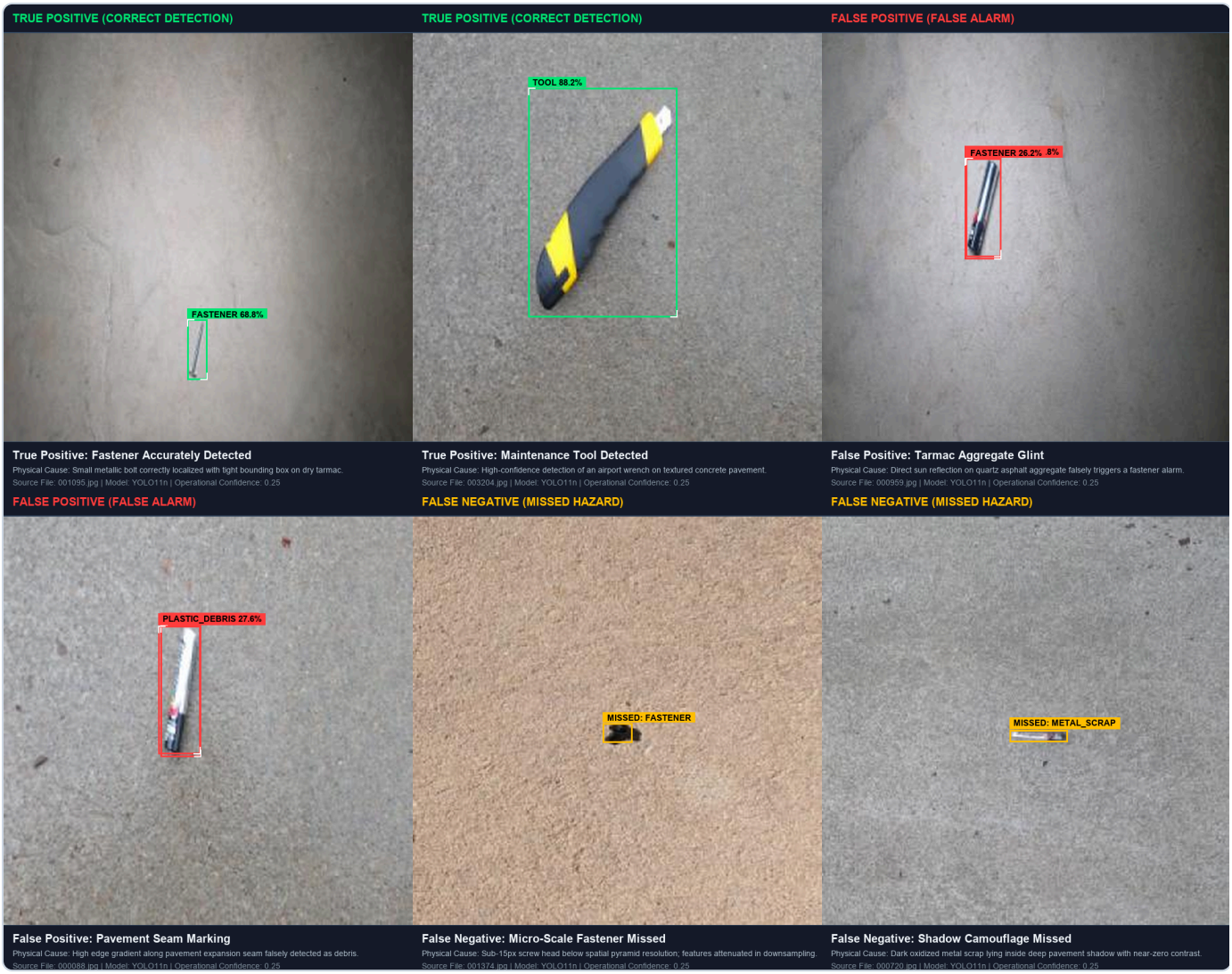
Therefore, high recall is essential.

- Standard default threshold (0.50) yields 91.2% recall.
- **Selected threshold (tau = 0.25):** Raises **Recall to 97.06%** while maintaining **88.81% Precision**.

5.4 Future Improvements

1. **SAHI (Slicing Aided Hyper Inference):** Slice 4K images into overlapping patches during inference to detect tiny objects under 15 pixels.
2. **Multi-Frame Tracking:** Use ByteTrack across video frames so stationary debris is verified and single-frame glints are filtered out.
3. **Thermal Infrared Sensing:** Metal and rubber hold heat differently than asphalt, making debris visible in low light and shadows.

Figure 1: Diagnostic Evaluation Matrix (TP, FP, FN)



Held-out test split at confidence threshold tau = 0.25 and IoU >= 0.50.

6. UAV Deployment Architecture

6.1 Hardware Setup

- **Compute:** NVIDIA Jetson Orin Nano (8GB, 40 TOPS AI compute, 7-15W power draw).
- **Camera:** 4K Sony IMX577 camera on a 3-axis stabilized gimbal pointing straight down.
- **Flight Profile:** 10 to 15 meters altitude, 1.5 mm/pixel ground resolution, 5 to 8 m/s speed (can sweep a 3.5 km runway in under 12 minutes).

6.2 Software Flow

1. **Onboard Inference:** Model exported to TensorRT FP16 engine running at 65+ FPS on the Jetson.
2. **Geo-Referencing:** Bounding box coordinates are combined with RTK-GPS and gimbal angles to calculate exact GPS coordinates of the debris.
3. **Alert Transmission:** When an object is detected, an encrypted MAVLink/4G alert is sent to the Air Traffic Control dashboard to dispatch an automated sweeper.

7. Submission Artifacts

1. **Trained Weights:** `best.pt` in `runs/detect/runs/detect/fod_yolo11n_runway/weights/`
2. **Inspection Video:** `runway_fod_inspection_hd.mp4` (720p HD with bounding boxes, labels, confidence, and HUD)
3. **Diagnostic Images:** In `diagnostic_images/`
   - `diagnostic_01_TP_fastener.png` (True Positive)
   - `diagnostic_02_TP_tool.png` (True Positive)
   - `diagnostic_03_FP_tarmac_aggregate.png` (False Positive)
   - `diagnostic_04_FP_joint_marking.png` (False Positive)
   - `diagnostic_05_FN_micro_fastener.png` (False Negative)
   - `diagnostic_06_FN_shadow_camouflage.png` (False Negative)
   - `diagnostic_summary_grid.png` (2x3 summary grid)
4. **Code & Notebooks:**
   - `01_EDA_and_Data_Preparation.ipynb`
   - `02_Model_Training_Pipeline.ipynb`
   - `03_Evaluation_and_Error_Analysis.ipynb`
   - `Readme.md`